

Object--Oriented Programming in Python

Classes and Objects

Object--Oriented Programming (OOP): A programming paradigm that involves designing programs around concepts represented as "objects"

- Python supports OOP through the provision of classes.
 - Class: A collection of functions and attributes, attached to a specific name, which represents an abstract concept.
 - Attribute: A named piece of data (i.e. variable associated with a class.
 - Object: A single concrete instance generated from a class

Instances of Classes

Classes can be viewed as factories or templates for generating new object instances.

Each object instance takes on the properties of the class from which it was created.

Instances of Classes

Abstract
concept



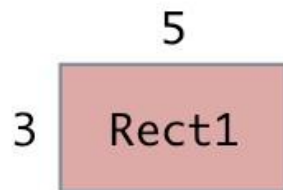
Attributes

- width
- height
- colour

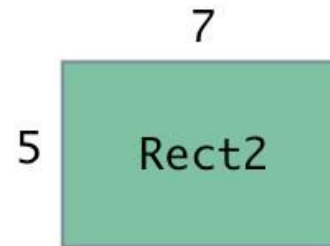
Functions

- area()

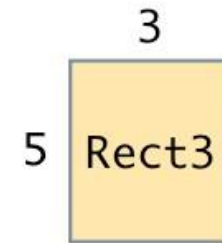
Multiple
concrete
instances



width=5
height=3
colour=(200,0,0)
area() -> 15



width=7
height=5
colour=(0,200,0)
area() -> 35



width=3
height=5
colour=(255,200,0)
area() -> 15

Creating Classes

Defining a class in Python is done using the class keyword, followed by an indented block with the class contents.

Class data
attributes



```
class <Classname>:
    data1 = value1
    ...
    dataM = valueM

    def <function1>(self, arg1, ..., argK):
        <block>
    ...
    def <functionN>(self, arg1, ..., argK):
        <block>
```

Class
member
functions



Defining Functions in Classes

- A class definition block can include multiple functions.
- These represent the functionality or behaviors that are associated with the class.

```
>>> class Maths:
...     def subtract(self, i, j):
...         return i-j
...
...     def add(self, x, y):
...         return x+y
```

Argument (self) refers to the object itself

Calling Functions in Classes

- Using Class Functions from Outside a Class
Functions are referenced by using the dot syntax:
<objectName>.<methodName>()

```
>>> m = Maths()  
>>> subtract(m, 10, 5)  
5  
>>> m.subtract(10, 5)  
5  
>>> m.add(6, 7)  
13
```



No need to
specify value for
self, Python does
this automatically

Calling Functions in Classes

- Using Class Functions from Inside a Class

When referring to functions from within a class, we must always prefix the function name with `self` (e.g. `self.subtract()`)

```
>>> class Maths:
...     def subtract(self,i,j):
...         return i-j
...
...     def testsub(self):
...         print self.subtract(8,4)
```

Tell Python to use function
associated with this object



Attributes

Class attribute
defined at top of
class

```
>>> class Person:
...     company = "ucd"
...
...     def __init__(self):
...         self.age = 23
```

Instance attribute
defined inside a class
function.
The `self` prefix is
always required.

```
>>> p1 = Person()
>>> p2 = Person()
>>> p1.age = 35
>>> print p2.age
23
```

Change to instance attribute `age`
affects only the associated
instance (`p2`)

```
>>> p1 = Person()
>>> p2 = Person()
>>> p1.company = "ibm"
>>> print p2.company
'ibm'
```

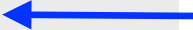
Change to class attribute `company`
affects all instances (`p1` and `p2`)

Constructor

- When an instance of a class is created, the class constructor function is automatically called.
- The constructor is always named `__init__()`
- It contains code for initializing a new instance of the class to a specific initial state (e.g. setting instance attribute values).

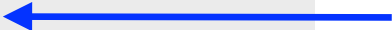
```
>>> class Person:
...     def __init__( self, s ):
...         self.name = s
...
...     def hello( self ):
...         print "Hello", self.name
```

Constructor function taking
initial value for instance
attribute name



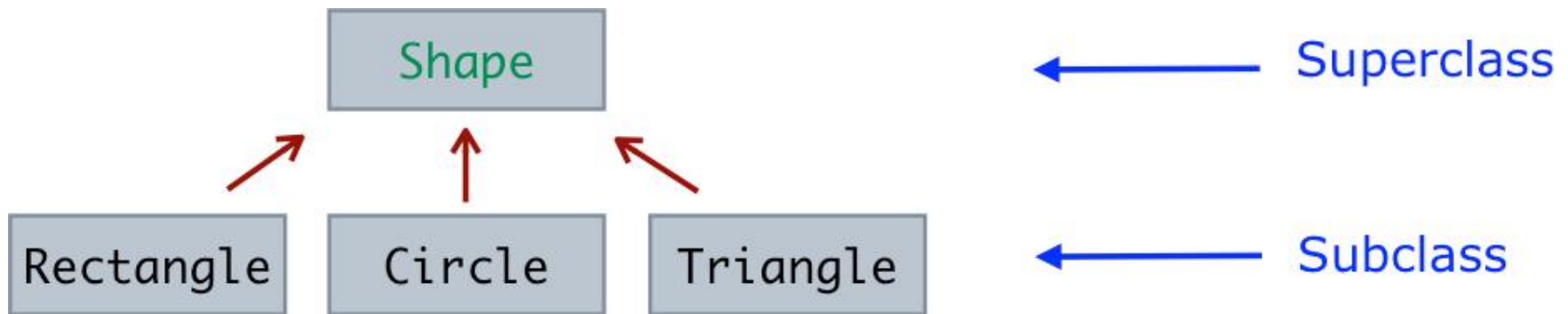
```
>>> t = Person("John")
>>> t.hello()
Hello John
```

Calls `__init__()`
on Person



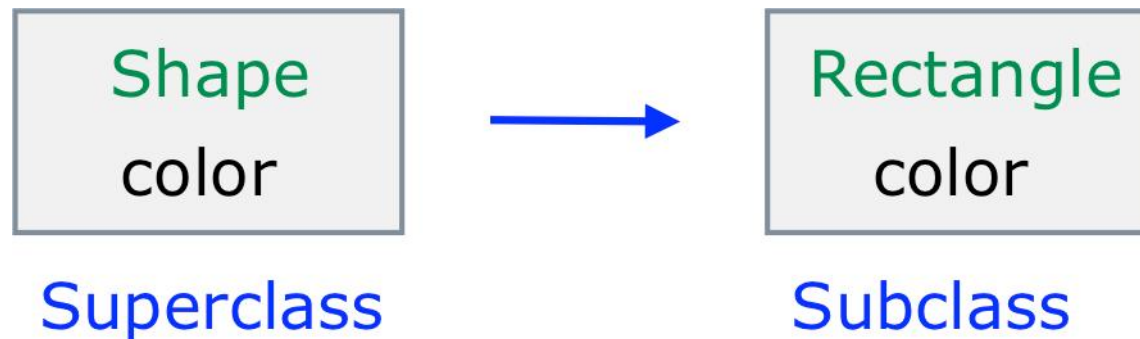
Inheritance

Class inheritance is designed to model relationships of the type "x is a y" (e.g. "a triangle is a shape")



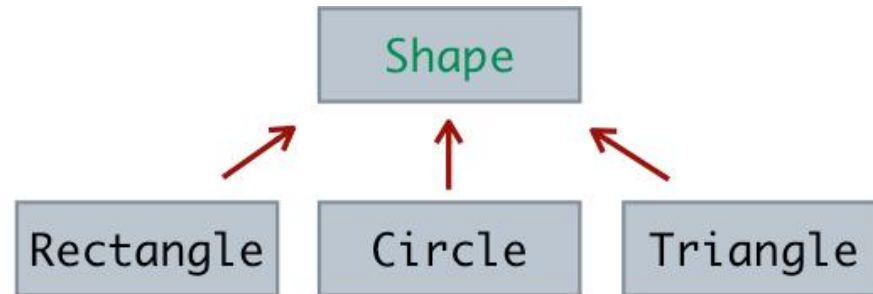
Inheritance

The functions and attributes of a superclass are inherited by a subclass.



An inherited class can override, modify or augment the functions and attributes of its parent class.

Creating Subclasses



Superclass



```
>>> class Shape:
...     pass
```

```
>>> Rectangle.__bases__
(<class '__main__.Shape'>,)
```

Name of superclass



Subclass



```
>>> class Rectangle(Shape):
...     def area(self):
...         return self.width*self.height
```

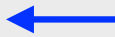
Simple superclass

```
>>> class Shape:
...     def __init__( self ):
...         self.color = (0,0,0)
```

Simple subclass inheriting from Shape

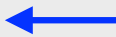
```
>>> class Rectangle(Shape):
...     def __init__( self, w, h ):
...         Shape.__init__( self )
...         self.width = w
...         self.height = h
...
...     def area( self ):
...         return self.width*self.height
```

Need to call
constructor
function in
superclass

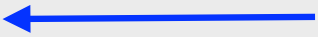


```
>>> r1 = Rectangle( 10, 5 )
>>> print r1.width
10
>>> print r1.height
5
>>> print r1.area()
50
>>> print r1.color
(0, 0, 0)
```

Construct
object instance



Inherited
attribute



Multiple Inheritance

A class can inherit from multiple parent classes.

```
class A:
    def action(self):
        print("Action from A")

class B:
    def action(self):
        print("Action from B")

class C(A, B):
    pass

>>> c = C()
>>> c.action() # Output: Action from A (MRO decides this)
>>> print(C.__mro__) # Output: (<class '__main__.C'>,
<class '__main__.A'>, <class '__main__.B'>, <class
'object'>)
```

Overriding

When inheriting from a class, we can alter the behavior of the original superclass by "overriding" functions (i.e. declaring functions in the subclass with the same name).

Functions in a subclass take precedence over functions in a superclass.

Overriding

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1
```

Overriding

```
class CustomCounter(Counter):
    def __init__(self, size):
        Counter.__init__(self)
        self.stepsize = size

    def increment(self):
        self.value += self.stepsize
```

```
>>> cc = CustomCounter(4)
>>> cc.increment()
>>> cc.print_current()
Current value is 4
```

Calls increment()
on CustomCounter
not Counter

Composition

Classes can be built from other smaller classes, allowing us to model relationships of the type "x **has a** y" (e.g. a department has students).

```
class Department:
    def __init__( self ):
        self.students = []

    def enroll( self, student ):
        self.students.append(student)
```

```
class Student:
    def __init__( self, last, first ):
        self.lastname = last
        self.firstname = first
```

```
>>> compsci = Department()
>>> compsci.enroll( Student( "Smith", "John" ) )
>>> compsci.enroll( Student( "Murphy", "Alice" ) )
```

Create Student
instances and add
to Department
instance



```
>>> for s in compsci.students:
...     print "%s %s" % (s.firstname,s.lastname)
...
John Smith
```

Encapsulation

Bundling data (attributes) and methods into a class and restricting access to some components.

- Attributes or methods are made private (hidden) by using a single (**`_attribute`**) or double (**`__attribute`**) underscore prefix.
- **Protected Attributes (`_attribute`)**: Use when you want the attribute to be accessible to subclasses but discourage external access.
- **Private Attributes (`__attribute`)**: Use when you want strict access control and want to restrict the attribute to the class only.
- Private attributes are accessed or modified through getter and setter methods.
- By hiding implementation details, encapsulation helps protect an object's state and ensures controlled interaction with it.

Encapsulation

```
class PublicExample:  
    def __init__(self):  
        self.name = "Public Attribute"
```

```
>>> obj = PublicExample()  
>>> print(obj.name)  ← # Accessible  
                        directly
```

```
class ProtectedExample:  
    def __init__(self):  
        self._name = "Protected Attribute"
```

```
>>> obj = ProtectedExample()  
>>> print(obj._name)  ← # Still  
                        accessible but  
                        discouraged
```

Encapsulation

```
class PrivateExample:
    def __init__(self):
        self.__name = "Private Attribute"

    def get_name(self):
        return self.__name
```

```
>>> obj = PrivateExample()
```

```
>>># print(obj.__name)
```

```
>>>print(obj.get_name())
```

#AttributeError:
Cannot access
private attribute

Use getter
method to access
private data

Feature	Protected Attribute (<code>_attribute</code>)	Private Attribute (<code>__attribute</code>)
Syntax	Single underscore (<code>_attribute</code>)	Double underscore (<code>__attribute</code>)
Access Control	Intended for internal use but not enforced.	Strictly limited to the class.
Subclass Access	Accessible in subclasses.	Not accessible in subclasses directly.
External Access	Can be accessed externally (discouraged).	Cannot be accessed directly.
Name Mangling	No name mangling.	Uses name mangling to prevent accidental access.

Name Mangling: Subclasses can access private attributes using the mangled name (`obj._ClassName__attribute`), but this is discouraged as it breaks encapsulation.

Interfaces

- An interface defines a contract that a class must follow. It specifies what methods a class must implement, but not how they should be implemented.
- In Python, there is no built-in interface keyword like in some other languages (e.g., Java). However, you can implement interfaces using **abstract base classes (ABCs)**, which are part of the abc module.

The ABC (Abstract Base Class) model in Python is a way to define abstract classes and enforce a blueprint for derived (or concrete) classes. Abstract Base Classes allow you to define methods that must be implemented in any subclass.


```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
```

```
    @abstractmethod
```

```
    def area(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def perimeter(self):
```

```
        pass
```

```
    def description(self):
```

```
        return "This is a shape."
```

```
class Rectangle(Shape):
```

```
    def __init__(self, width, height):
```

```
        self.width = width
```

```
        self.height = height
```

```
    def area(self):
```

```
        return self.width * self.height
```

```
    def perimeter(self):
```

```
        return 2 * (self.width + self.height)
```

Polymorphism

Two objects of different classes but supporting the same set of functions or attributes can be treated identically.

The implementations may differ internally, but the outward "appearance" is the same.

- Python supports polymorphism with built-in functions and operators, like **len()** or **+**.

Polymorphism

1. Polymorphism via Method Overriding

Two different classes that contain the function **area()**

```
class Rectangle(Shape):
    def __init__(self, w, h):
        Shape.__init__(self)
        self.width = w
        self.height = h

    def area(self):
        return self.width*self.height
```

```
class Circle(Shape):
    def __init__(self, rad):
        Shape.__init__(self)
        self.radius = rad

    def area(self):
        return math.pi*(self.radius**2)
```

2. via Interfaces or Abstract Classes

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

Instances of the two classes can be treated identically...

```
>>> l = []
>>> l.append( Rectangle(4,5) )
>>> l.append( Circle(3) )
>>> for someshape in l:
...     print someshape.area()
...
20 Result of area() in Rectangle
28.2743338823 Result of area() in Circle
```

Static and Class Methods

- A **static method** does not depend on the class or instance data. It is used for utility or helper functions.
- A **class method** works with the class itself (cls) and can access or modify class-level attributes.

```
class MathUtils:
```

```
    @staticmethod
```

```
    def add(x, y):  
        return x + y
```

```
>>> print(MathUtils.add(3, 5)) # Output: 8
```

we can call it
directly without
creating an object.

modify the class-level
attributes `pi` using
the `cls`

```
class Circle:
```

```
    pi = 3.14 # Class-level attribute
```

```
    @classmethod
```

```
    def change_pi(cls, new_pi):  
        cls.pi = new_pi
```

```
>>> Circle.change_pi(3.14159)
```

```
>>> print(Circle.pi) # Output: 3.14159
```

Difference Between Instance, Static, and Class Methods

1. Instance Method (Simple Method):

- Requires an instance of the class to be called.
- Takes **self** as its first parameter.
- Can access and modify both instance and class-level attributes.⁷

2. Static Method:

- Does not require an instance or class to operate.
- Takes no self or cls parameter.
- Cannot access instance or class-level attributes.
- Used for utility or helper functions that are logically related to the class but do not interact with its data.

3. Class Method:

- Requires the class itself to be passed as the first argument (cls).
- Takes **cls** as its first parameter.
- Can access and modify class-level attributes but not instance-specific attributes.
- Often used for factory methods or for behavior that affects the entire class.

Enumeration

An enumeration (or enum) is a way to define a set of named constants that belong together. Enums are useful when you have a fixed number of possible values for a variable.

```
from enum import Enum

class Day(Enum):
    MONDAY = 1
    TUESDAY = 2
    WEDNESDAY = 3
    THURSDAY = 4
    FRIDAY = 5
    SATURDAY = 6
    SUNDAY = 7

>>> print(Day.MONDAY.name)
# Output: MONDAY
>>> print(Day.MONDAY.value)
# Output: 1
```

Magic/Dunder Methods

- Magic methods (also called dunder methods) are special methods in Python with double underscores (__) at the beginning and end of their names. For example, **`__init__`**, **`__str__`**, and **`__eq__`**.
- These methods provide a way to define custom behavior for objects in Python, such as **initializing, printing, comparing, or performing arithmetic operations.**

Commonly Used Dunder Methods

- **__init__**: Called when an object is created. Used to initialize the object's attributes.
- **__str__**: Defines the string representation of an object (used when **print()** is called on the object).

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):
        return f"{self.title} by {self.author}"

>>> book = Book("1984", "George Orwell")
>>> print(book) # Output: 1984 by
George Orwell
```

__repr__: Defines a developer-friendly representation of the object (used in debugging). Difference from **__str__** . **__str__** is **user-focused**, while **__repr__** is **developer-focused**.

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __repr__(self):
        return f"Book(title='{self.title}', author='{self.author}')"

>>> book = Book("1984", "George Orwell")
>>> print(repr(book)) # Output: Book(title='1984',
author='George Orwell')
```

__eq__: Defines how objects are compared using ==.

```
class Book:
    def __init__(self, title, isbn):
        self.title = title
        self.isbn = isbn

    def __eq__(self, other):
        return self.isbn == other.isbn
```

```
>>> book1 = Book("1984", "12345")
>>> book2 = Book("Animal Farm",
"12345")
>>> print(book1 == book2)
# Output: True (compares ISBNs)
```

__add__: Defines the behavior for the + operator.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y +
other.y)
```

```
>>> p1 = Point(2, 3)
>>> p2 = Point(4, 5)
>>> result = p1 + p2
>>> print(result.x, result.y) # Output: 6 8
```

`__del__(self)`: Executes when an object is destroyed. Used to free resources, such as closing files or connections.

```
class Demo:
    def __del__(self):
        print("Objet détruit")

d = Demo()
del d  # Affiche "Objet détruit"
```

Résumé des Méthodes Magiques Fréquemment Utilisées

Méthode	Description	Exemple d'Utilisation
<code>__init__</code>	Initialisation d'un objet	<code>obj = MyClass()</code>
<code>__str__</code>	Représentation conviviale	<code>print(obj)</code>
<code>__repr__</code>	Représentation technique	<code>repr(obj)</code>
<code>__add__</code>	Surcharge de l'opérateur +	<code>obj1 + obj2</code>
<code>__len__</code>	Taille de l'objet	<code>len(obj)</code>
<code>__getitem__</code>	Accès par index/clé	<code>obj[key]</code>
<code>__setitem__</code>	Modification par index/clé	<code>obj[key] = value</code>
<code>__enter__</code>	Entrée dans un contexte (with)	<code>with obj:</code>
<code>__exit__</code>	Sortie d'un contexte (with)	<code>with obj:</code>

Autres méthodes d'opérateurs courants :

- `__sub__(self, other)` : Pour - (soustraction).
- `__mul__(self, other)` : Pour * (multiplication).
- `__truediv__(self, other)` : Pour / (division).
- `__eq__(self, other)` : Pour == (égalité).
- `__lt__(self, other)` : Pour < (inférieur).
- `__gt__(self, other)` : Pour > (supérieur).